Multi-Channel Hebbian Plasticity in Multilayer BDH: A Learned Partition of Working Memory for Context-Conditional Binding

A window gate, slow memory and a targeted channel split find an eight-way partition of a synthetic binding task
without stream labels or restarts, on two machines with disjoint seeds, and in an independent replication

A technical report • Revision 9 (draft)

Project repository: `Ynkling/clankers`

Abstract

BDH's Hebbian working memory is split into k channels, and a per-token gate distributes each token's write over the channels and blends its read from them. On a synthetic task, S streams bind the same keys to different values, cued by a stream token; one channel cannot hold the conflicting bindings at four or eight streams, and a hand-set gate, one channel per stream, can. The question is whether a gate trained only on the task loss finds that partition. Revision 8 answered it for the task's grouped layout, where the cue is within three tokens of every write: a *window gate* (a width-3 causal convolution of the embeddings), slow memory for the first 2400 updates and a plateau-triggered *channel split* that copies the busiest channel's gate row onto the idlest bind eight streams, the streams on disjoint sets of channels, in 38/40 runs of a pre-registered test on two machines with disjoint seeds, against 0/40 for the recurrent-gate recipe (38 vs 0 discordant pairs) and 15/40 without the split (23 vs 0). The recipe uses no stream labels, no restarts and no hinge; the split reads the task's key positions.

This revision reports what has been done to that result since. An independent referee session recomputed every number in the report from the records (all reproduce), audited the pre-registration of every claim (none edited after a run), compiled the project's claim registry (53 pre-registered tests per machine, of which the headline survives any multiplicity correction), and found statements in Revision 8 that were wrong when committed; they are corrected and listed. A clean-room replication from the report alone, by a session that never saw the code, bound eight streams 17/20 against 7/20 without the split (10 vs 0): the model it built agrees with ours to 10^{-8}, the 44 details it had to guess are now stated, and one of them exposed a sampling weakness in the split's own probe. On the main line, the second claim of Revision 8 is now a pre-registered result on one machine (the other's half is running): the split's row copy is the ingredient and its optimizer reset alone is not enough (copy and reset 10/10 against the reset alone 3/10, 7 vs 0; the reset alone no better than no trigger), and one delta-rule channel binds eight streams 0/10 where the partition binds 10/10. Exploratory screens settled three questions the earlier revision left open. The delta-rule write, with its write strength fixed rather than learned, matches the Hebbian write at two, four and eight streams and binds sooner, so the memory rule is open again and is the next main-line test. On Hebbian memory the layer whose routing matters is the first: perfect gates at the later layers alone bind 0/10, and a register that overwrites at the context token, read by a gate at all three layers, carries a cue up to six tokens away 20/20 when nudged by labels, is found label-free on 4/20 seeds, the failures being key splits that form first, and is found 20/20 when a hinge on the gate's key-dependence holds the key split off for the first 2400 updates, a training-time prior that reads the task's key positions and identities as the split does. And under a dense next-token loss on interleaved sources, a single channel learns to carry the source by itself. Two-stream key splits resist every repair tried; a replication's probe can be wrong about a plateau; the title is scoped to what was shown.

Revision 8 (7–10 October 2026). Revision 8 added Phase VI: two pre-registered tests on two machines with disjoint seeds, an audit of the project's records (`docs/audit_2026-10.md`), five exploratory batches on the side branch, three exploratory sessions on their own branches and a reading of twelve papers (`docs/reading/`). Its first draft was reviewed by an independent referee session, which found statements that were wrong when committed; Revision 8.1 corrected them and nothing else (the list is in Section 15).

Revision 9 (draft). This revision adds, on the main line, a third pre-registered test (`test_split_copy`, complete on X and running on L) and the specification of a fourth (`test_delta_recipe`); the referee's report, recomputation and claim registry, merged into the repository (`docs/review_2026-10/`); an independent clean-room replication of the main result (branch `claude/replicate-R`); the exploratory sessions' second and third rounds (F's S68–S81, G's S55–S82, H's S61–S85, I's S74 and a GPU package); and L's result files for Phases III–IV, which were missing from the repository. Two machines ran every main-line test: **X**, a cloud container on Intel Xeon hosts at 2.10 and 2.80 GHz, and **L**, an Intel i7-12650H. From the Muon-recipe test on, X and L run disjoint seeds and the pooled claim is primary. The exploratory sessions are **E** (branch `claude/outside-ideas`, batches 1–19), **F**, **G**, **H** and **I** (branches `claude/explore-F/G/H/I`), the replication **R** (`claude/replicate-R`) and the referee **V** (`claude/review-V`); F, G, H, I, R and V ran on 2.10 GHz Xeons, E's batches mostly on 2.10 GHz hosts with 60 of batch 16's 165 runs and batch 19 on 2.80 GHz. Every verdict was fixed before its run and printed mechanically; the referee's recomputation (Section 15) found every pre-registered reading of every screen to re-derive from its records. The revision is a draft: L's half of `test_split_copy`, E's batch 19 (S66–S67), F's S81, H's S85 and R's Phase 4 are running, and everything remains one small model on synthetic tasks. The title has been scoped to what was shown.

test, batch or session	question	verdict	§
`muon_recipe` (X, L, pooled)	the early hinge window under Muon; Muon's speed at four streams and eight keys	M1 SHOWN pooled; M2, M3 NOT SHOWN pooled; reading not met	7
`window_gate` (X, L, pooled)	the window gate with slow memory, no hinge, at 2, 4 and 8 streams; the split at 8	G1, G2, G4 SHOWN pooled; G3 HOLDS; "eight streams bind without labels or restarts"	5
`split_copy` (X; L running)	the copy against the reset; one delta channel at eight streams	on X: C1, C3 SHOWN, C2 HOLDS; pooled readings wait for L	6, 11
`delta_recipe` (specified)	the delta write against the Hebbian write at eight streams	spec under referee review; not run	11
batches 15–18 (E)	the recurrent gate's cap; the window gate; the split; the reset control; key splits	as in Revision 8	4, 6
batch 19 (E, running)	width and depth at four streams; eight streams at the larger sizes	scale-free to $N = 1024$, $D = 128$, 4 layers at four streams	10
session F	delta-rule channels; the premise test; forget gates; the lite task; the probe; the delta recipe	one delta channel 2/10, 0/10 at four and eight streams; the delta recipe with fixed β keeps 2, 4 and 8 streams	11
session G	the cue in the stack; the register; which layers must be routed	the cue is absent; layer 1 is the block on Hebbian memory; the register carries the cue 20/20 nudged, 4/20 label-free, 20/20 with the key hinge	12
session H	anti-collapse devices; the copy without reset; the split's constants; two streams	nothing replaces the split; the copy alone suffices; the cap matters; the flat-start guard aims the splits without changing the count; two-stream key splits resist the copy	6, 9
session I	a dense next-token loss on interleaved sources	one channel learns to carry the source; four sources need a GPU	19
session R	a clean-room replication of `window_gate`'s eight-stream result	17/20 against 7/20 (10 vs 0): partially replicated by its own bar, consistent with 38/40	14
session V	referee: recomputation, pre-registration audit, claim registry, literature	all numbers reproduce; 53 tests per machine; Revision 8's errors; the probe's imbalance	15

Table 1: Phase VI. Main-line tests are files `test_<name>.py` whose docstring records the pre-registered design and, after the run, the result on each machine and pooled; from `test_window_gate` on, the specification is committed to `specs/` before the test is written, and from `test_delta_recipe` on it is reviewed by the referee session before any run. Exploratory batches and sessions write only new files on their own branches and label every output "exploratory, not a result". Appendix A lists the commits.

1 Background

BDH [1] stores working memory in synaptic state updated by a Hebbian rule. In its GPU form the operation is linear attention: the score between a query position t and a past position s is a content match $x_t \cdot x_s$ between sparse positive neuron activations, weighted by a positional operator (scalar decay or RoPE). The public implementation, `bdh.py`, ties queries to keys ($Q = K$), shares one layer's weights across depth, and LayerNorms the attention output. Its state update, in the paper's notation, is $\rho_t = \rho_{t-1} + \mathrm{LN}(Ey_t)\, x_t^\top U$: a pure additive write.

The extension gives every synapse k channels and every token a gate $g_t \in \Delta^{k-1}$ that distributes its write over the channels and blends its read from them. With one distribution for both, the

effective score becomes

$$\underbrace{(x_t \cdot x_s)}_{\text{content match}} \times \underbrace{(g_t \cdot g_s)}_{\text{context-gate match}} \times \text{decay}^{\,t-s}, \tag{1}$$

strictly causal ($s < t$), so tokens sharing a context can read each other's memory while tokens in different contexts are isolated. The score has the three-factor form that Triadic Linear Attention [3] describes (cited for its form; the paper is not among the twelve read in full), with g as the second key; there the second key is a learned projection of the token, here it is a routing distribution inferred from context, and the subject is its label-free discovery. With $k = 1$ the model is single-channel BDH. One gate g_t is computed per token from the raw token embedding v_t (not LayerNormed, not convolved, not the residual stream) and applied at every layer. The hand-set *perfect gate* sends every token of stream s to channel s; it is used only as a validity ceiling.

Two learned gates. The *recurrent gate* of Phases II–V,

$$h_t = \tanh\big(W_{\text{in}}\, v_t + W_h\, h_{t-1}\big), \qquad g_t = \text{softmax}(W_g\, h_t), \tag{2}$$

with 32 units in h_t and $W_{\text{in}}, W_h, W_g \sim \mathcal{N}(0, 0.1^2)$ (so that $\rho(W_h) \approx 0.6$ and $\sigma_{\max}(W_h) \approx 1.08$ at initialization). The *window gate* of this revision,

$$h_t = \tanh\Big(W_{\text{in}} \sum_{j<3} w_j \odot v_{t-j}\Big), \qquad g_t = \text{softmax}(W_g\, h_t), \tag{3}$$

a depthwise causal convolution of width 3 over the embeddings (weights drawn uniformly in $\pm 1/\sqrt{3}$, no bias), with no recurrence. It sees the current token and the two before it, which in the grouped layout is enough to see each key's stream token at the key, the value and the query. It cannot see anything further back, and it has no notion of position.

The uniform gate is a stationary point. Writing each gate as a deviation from uniform, $g_t = \frac{1}{k}\mathbf{1} + \delta_t$, gives $g_t \cdot g_s = \frac{1}{k} + \delta_t \cdot \delta_s$. The constant is cancelled by the LayerNorm after attention, so routing enters only through products of deviations, and at the uniform gate the task loss exerts no first-order pull toward any routing. This is why discovery is a race between the gate's first commitment and whatever the memory learns first, and why the recipes below act in the first few thousand updates.

A short causal convolution. Most arms include the short convolution that Mamba and most linear-attention models place before their token mixer [4]: a depthwise causal filter of width 4 at every layer, initialized to the identity, feeding the queries and the values. Without it, the perfect gate of this model ($N = 256$) binds the two-stream configuration on every seed recorded (4/4 in the main line's CEIL_A, 10/10 in session F). The first draft attributed four failures to the missing convolution; those runs used a smaller model (`test_multilayer_binding`'s $N = 64$ ceiling, 7,552 parameters against 25,984), as the referee's recount found. Whether the convolution carries part of the binding is open.

2 Earlier phases in brief

Phases I–IV (Revisions 1–6). A gate that was a function of token identity could not express the routing (Phase I); a recurrent gate could (Phase II). On the binding task of Section 3, two channels with the perfect gate bind two streams where no single-channel model binds, and a label-free learned

gate finds the routing on about half its runs (Phase III). A restart rule made two streams reliable; four streams with four channels mostly failed by merging streams into a shared channel, and spare channels removed most merges (Phase IV).

Phase V (Revision 7). An early-training recipe replaced restarts at two streams and at four with sixteen channels: for the first 2400 updates the memory trains at a tenth of the gate's rate (**SLOW**), and a hinge penalizes the gate when its channel choice at key positions is explained by position (**HINGE**; **WINDOW** switches the hinge off after update 2400). Eight streams failed in every arm (none of more than 100 runs), and screens located the failure in the recurrent gate: its state carries the stream at initialization and loses it within the first 600–1200 updates, as its recurrent term grows and, in seven of ten runs, the spectral radius of W_h rises above one. Revision 7 ended with a cap on that recurrence being screened.

3 The tasks, the outcomes and the recipes

The binding task. There are S streams, each with a context token CTX_s. For each of P keys, S distinct values are drawn from 16 value tokens, so every key is bound to a different value in every stream. In the *grouped* layout the body is SP triples $[\mathrm{CTX}_s, \mathrm{KEY}_i, \mathrm{VAL}_{i,s}]$, grouped by key, keys in random order and stream order random within each key's group. One query $[\mathrm{CTX}_q, \mathrm{KEY}_q, ?]$ ends the sequence; the model predicts the value from the query's KEY position, where CTX_q is one token back. The write that must carry the stream is at the VAL position, where CTX is two tokens back. Chance among values is 1/16; a model that binds keys to values but cannot tell streams apart scores $1/S$. The *header* layout gives each stream one block $[\mathrm{CTX}_s, \mathrm{KEY}, \mathrm{VAL}, \mathrm{KEY}, \mathrm{VAL}, \ldots]$ with the stream token once at the start, so most writes are far from their cue; at a fixed block length, position can route it. The *randomised header layout* ("RandHeaderTask-lite", session F's S70; the distant-cue task of Sections 12 and 14) has $S = 2$ streams of $P = 4$ keys, each stream's keys in two blocks whose lengths are a uniformly drawn composition of 4 into parts of 1–3 pairs, the four blocks in uniformly random order, one CTX token at the start of each block, every (stream, key) once, and one query; $L = 23$, and the cue is one to six tokens from the write it governs. Both perfect gates bind it by update 1200 on five of six seeds. The model has three layers with weights shared across depth, decay 0.95 (half-life about 14 tokens), $N = 256$, embedding width 32 and one head, and trains at batch 32 for a budget set per test (24,000, 28,800 or 43,200 updates), with evaluation every 1200 updates on 2048 held-out queries; a run stops early after three consecutive evaluations at or above 0.95. Session I's dense-loss task is defined in Section 19.

Outcomes.

- **Bound:** an evaluation reaches 0.95 and every later one stays there; the *transition* is the first such step. A run that reaches its budget counts as bound on a single final evaluation; this has happened in 14 of 1869 bound runs in `results/` at Revision 8's commit (the referee's recount).
- **Discovered** ($k = S = 2$): bound, with the streams' read gates separated at value positions (final VAL cosine below 0.5).
- **Bound routed** ($k \geq S > 2$): bound, every stream has its own channel (a one-to-one *stream-to-channel map*, measured at value positions, where the writes are), and every stream's accuracy is at least 0.9. The map takes each stream's largest-mass channel, from a 256-sequence probe, ties resolved on unrounded values; in the 38 headline runs the streams' channel sets are disjoint

(largest pairwise overlap 0.047) while in 18 of them some stream's largest channel holds under 0.9 of its gate mass. On the randomised header layout session F's **ROUTED*@end** (routing margin ≥ 0.9 and η^2 by stream at key positions > 0.9 at the end) is used, and the readings of Sections 12 and 14 are on **ROUTED-BOUND**, bound and ROUTED*@end; the name DISCOVERED is reserved for the outcome above, since one screen's reading named "discovered" was fixed on ROUTED-BOUND and would have come out differently on DISCOVERED (Section 12).

- **Failure classes:** POSITION or KEY when the gate's channel choice at key positions is explained ($\eta^2 \geq 0.5$) by position or by key identity; STREAM-PARTIAL; OTHER. For $S > 2$: MERGED (j share) when j streams share a channel, non-stream when the gate does not split by stream, *collapsed* when final accuracy is below 0.15. The classifier of record (v1) applies the margin rule before the key rule and labelled three batch-18 runs at exactly $\eta^2 = 0.5$ by key STREAM-PARTIAL; `fail_class_v2` (commit `ba8901a`) applies the key rule first and is used by every test from `test_split_copy` on; recorded verdicts keep the rule they were read under, and the two classifiers disagree only where η^2 by key ≥ 0.5 and the margin rule fired (six of the 29 recorded unbound two-stream runs on H's branch, four of S78's seventeen, and none at eight streams in S79).

Recipes.

- **SLOW:** one Adam optimizer with two groups. The gate's parameters train at 10^{-3} throughout; every other parameter, embedding and convolution included, at 10^{-4} for updates 1–2400 and 10^{-3} after. The perfect gate (the validity arm) trains at a single rate, 10^{-3}, without SLOW.
- **HINGE, WINDOW:** Revision 7's position hinge, on the recurrent gate; kept here only as the comparison.
- **SPLIT (KEYMASS):** at checks every 2400 updates from 4800, if the probe's accuracy is below 0.95 and rose by less than 0.02 since the previous check (the probe is also read at 2400 as the first reference; the last check is at the budget minus 2400), copy the gate row $W_g[c^*]$ of the channel with the largest mean read-gate mass at the body's key positions onto the row $W_g[c_0]$ of the channel with the smallest; at most three splits per run, at least 4800 updates apart. `test_window_gate`'s form adds noise of 0.1 times the row's standard deviation to both rows and zeroes W_g's Adam state, step counter included; session H's *exact copy* does neither, and is the form of the screens of Sections 6 and 11 and of the next main-line test. The probe is one fixed set of 64 sequences, the same for every seed and arm (`Generator(12345)`), scored on its queries only; its queries fall 5, 5, 6, 8, 10, 12, 5 and 13 on streams 0–7, so it is unbalanced in the same way for every run (Section 14). The split uses no stream labels; it needs to know where the key positions are.
- **MUON:** Muon at learning rate 0.005 on every 2-D weight (momentum 0.95, Nesterov), with Adam on the rest, and SLOW's schedule.
- **The delta write (session F):** each channel's state follows Gated DeltaNet's rule, $S_t^c = S_{t-1}^c\big(\alpha_t^c(I - g_t^c\beta_t\hat{x}_t\hat{x}_t^\top)\big) + g_t^c\beta_t v_t\hat{x}_t^\top$, with unit keys $\hat{x}_t$, the decay fixed at 0.95, and the write strength β *fixed at 1*; at $\beta = 0$ its logits agree with the Hebbian model's to 10^{-7}, and with the new path disabled it reproduces a recorded run bit for bit. A learned β collapses toward "do not write" (Section 11).
- **The register gate (session G):**a 16-dimensional register $R_t = R_{t-1}(1 - \beta_t^R) + \beta_t^R W_R v_t$, $\beta_t^R = \sigma(w_R \cdot v_t + b_R)$ with b_R initialised to logit 0.1, written by every token and

read causally (the gate at t reads R_{t-1}); one gate shared by all three layers, $g_t^{(\ell)} = \mathrm{softmax}(W_g \tanh(A\,[\mathrm{LN}(r_t^{(\ell)});\, u_t;\, R_{t-1}]))$ over the layer's residual input, the window gate's input u_t and the register, with A initialised so that the gate equals the window gate at step 0, and a stop-gradient from the gate into the stack. "REG3" is this gate at all three layers; Revision 8's design had it at layers 2–3 only, with the window gate at layer 1.

Statistics. Paired comparisons in the main-line tests use exact one-sided McNemar tests on the discordant pairs ("b vs c"), unpaired ones Fisher's exact test; E's exploratory batches report two-sided McNemar p-values and sessions F, G and H one-sided ones, as marked where quoted; bands are RELIABLE at $\geq 90\%$ and MAJORITY at $\geq 50\%$ of runs, with Wilson 95% intervals. From the Muon-recipe test on, X and L run disjoint seeds (X 400–599, L 1400–1599), the pooled claim over both machines' pairs is primary and each machine's is secondary. The referee's claim registry (`docs/review_2026-10/claims_registry.md`) counts 53 pre-registered hypothesis tests per machine over Phases III–VI (52 one-sided; 25 Fisher, 25 McNemar, 2 Mann–Whitney, 1 sign test), plus six pooled tests in Phase VI, besides 3 bounds without a p-value, 41 bands, 29 validity gates and 37 readings; every test was run at $\alpha = 0.05$ with no adjustment, which controls the per-comparison rate only, so about 2.7 false positives per machine are expected if every null were true. The project controls the false-discovery rate: under Benjamini–Hochberg at 0.05, 19 of X's 23 positive verdicts and 27 of L's 32 survive, and the four on X that depend on the unadjusted level (`stream_recipe` R3, `recipe_scope` Q2, `slow_start` S0, `muon_recipe` M2) carry no conclusion below; the main result (G2, G4; the pooled G1 and M1) survives Holm on X, on L and pooled. The conclusions that matter are those replicated on both machines or pooled over disjoint seeds.

4 The window gate

History. The window gate was the first screen of the first exploratory batch (S1, batch 1): on seeds 160–179 with plain Adam and no recipe it discovered 15/20 against the recurrent gate's 7/20 (12 vs 4, two-sided $p = 0.077$: "promising" by the screen's rule) and bound sooner on average (mean transition 2720 against 3429; equal medians); its five failures were three key splits and two stream-partial runs under the main-line classifier, four key splits under the key-first rule (it cannot position-split, having no position). Batch 2 ran it on the header layout, where it bound 2/10, both by key splits, and it was dropped: a gate that cannot see a cue three tokens back seemed the wrong direction for a language model. It was never combined with slow memory and never run at four or eight streams. The October audit of the project's records (`docs/audit_2026-10.md`, Section 3.1) noticed that every obstacle of Phase V was a property of the recurrent parameterization: position splits are the recurrence counting triples, and the eight-stream loss of the stream is the recurrence becoming expansive. A window gate can do neither. The audit fixed a prediction before any run: with slow memory, at two streams $\geq 36/40$ with key failures gone; at eight streams, binding on some seeds where every recurrent arm bound none.

Batch 16 (E). The window gate with SLOW and no hinge (LOCAL3_SLOW): at two streams 31/40 on seeds 160–199 (STREAM-PARTIAL 5, KEY 4, no POSITION), inconclusive against the hinge recipe's 34/40 (5 vs 8). At four streams with sixteen channels, 20/20 bound routed against X's recorded HINGE4k16 15/20 (5 vs 0, two-sided $p = 0.0625$). At eight streams, 3/10 under Adam and 3/10 under Muon, where every recurrent arm in batches 13–16 had bound 0/10; the seven unbound Adam runs ended with two or three streams sharing a channel at accuracy 0.73–0.88. The window gate's state decodes the stream at both key and value positions (1.00 at four streams at

update 4800, the two runs still training at 9600 reading 0.70–0.72; at eight, 0.93–1.00 under Muon and, under Adam, 0.55–0.67 at updates 2400–4800 recovering to 0.87–0.88 by 9600), where the recurrent gate's reads 0.14–0.28. So the audit's first prediction was partly met (no position failures; key failures reduced, not gone, and 31/40 short of 36), and its second was met.

Batch 17 (E). The failures at eight streams were merges, so batch 17 added the targeted split of Section 3 (S37's KEYMASS rule, at most three splits). W_SPLIT bound routed 9/10 against 3/10 for the same seeds without it (6 vs 0, two-sided $p = 0.031$); every one of its 19 splits was on target by the labels (the busy channel held two to five streams, the idle one none), and in six of nine bound runs the transition came at the evaluation right after the last split. Decay 0.98 alone bound 1/10 (it settled the merges early) and a longer budget alone 4/10: the splits, not the decay or the time, do the work. The screen handed W_SPLIT back for a pre-registered test.

5 The main result: `test_window_gate`

The test's specification was committed before the test was written (`specs/test_window_gate.md`) and amended, before any Part D run, to add the split. Both machines ran it on their own seeds (X 500–559, L 1500–1559). Part A ($S = 2$, $P = 4$, $k = 2$, no convolution, 24,000 updates, 40 seeds per machine): the window gate with one learning rate (WIN3), with slow memory (WIN3_SLOW), and the recurrent-gate recipe (HINGE0). Part B ($S = 4$, $P = 4$, $k = 16$, convolution, 28,800, 20 seeds): WIN3_SLOW16 against WIN16_A, the recurrent gate under Adam with the early hinge window. Parts C and D ($S = 8$, $P = 4$, $k = 16$, convolution, 43,200, 20 seeds): WIN3_SLOW_D8, HINGE_D8 (the recurrent recipe) and WIN3_SPLIT_D8 (the window gate with the split), with a reset-only control (WIN3_RESET_D8: the Adam-state reset at the same trigger, no copy) on five seeds per machine, printed and not claimed. The perfect gate on two seeds per part was the validity arm, and bound on every one (Part C at 4800 and 3600–6000). Every CHECK passed on X, including bit-for-bit reproductions of batch 1's window gate, batch 16's LOCAL3_SLOW and batch 17's W_SPLIT (its split at 4800 included); L's file carries the metadata the test writes only after its verification passes. Nothing was cut; X ran its 225 learned-gate records in 30.9 h of training (its six perfect-gate records came from an earlier start that a container restart ended), L its 231 records in one start (projected 17.3 h).

part	arm	X	L	pooled
A: S=2, P=4, k=2, discovered	WIN3	33/40	32/40	65/80
	WIN3_SLOW	33/40	36/40	69/80
	HINGE0 (recurrent, Rev. 7's recipe)	33/40	34/40	67/80
B: S=4, k=16, bound routed	WIN3_SLOW16	19/20	19/20	38/40
	WIN16_A (recurrent, early window)	15/20	12/20	27/40
C, D: S=8, k=16, bound routed	WIN3_SPLIT_D8 (the recipe)	18/20	20/20	**38/40**
	WIN3_SLOW_D8 (no split)	6/20	9/20	15/40
	HINGE_D8 (recurrent, Rev. 7's recipe)	0/20	0/20	0/40
	WIN3_RESET_D8 (control, of 5)	0/5	3/5	3/10
	perfect gate (of 2)	2/2	2/2	4/4

Table 2: `test_window_gate`. Pooled claims (primary, exact one-sided McNemar over both machines' pairs): G1, WIN3_SLOW16 beats WIN16_A: 13 vs 2 (X 5 vs 1, L 8 vs 1), $p = 0.0037$, SHOWN. G2, WIN3_SPLIT_D8 beats HINGE_D8: 38 vs 0, $p = 3.6 \times 10^{-12}$, SHOWN. G3, WIN3_SLOW not worse than HINGE0 by more than two discordant pairs: HINGE0 only 9, WIN3_SLOW only 11, $d = -2$, HOLDS (a bound, no p-value). G4, WIN3_SPLIT_D8 beats WIN3_SLOW_D8: 23 vs 0 (X 12 vs 0, L 11 vs 0), $p = 1.2 \times 10^{-7}$, SHOWN. Per machine every claim is SHOWN or HOLDS on both, except G1 on X (5 vs 1, $p = 0.11$; on L 8 vs 1, $p = 0.020$). Bands, pooled: WIN3_SPLIT_D8 RELIABLE, Wilson $[0.835, 0.986]$; WIN3_SLOW16 RELIABLE $[0.835, 0.986]$; WIN3_SLOW MAJORITY $[0.770, 0.921]$; WIN3_SLOW_D8 MINORITY $[0.242, 0.530]$; HINGE_D8 NEVER $[0, 0.088]$. The pre-registered reading, "eight streams bind without labels or restarts" (WIN3_SPLIT_D8 RELIABLE pooled and G2, G4 SHOWN), applies.

What the runs show (diagnostics, not claims). The recurrent-gate recipe bound none of 40 eight-stream runs (15 collapsed, 17 non-stream, 8 merged). The window gate alone bound 15/40, and every one of its 25 failures was a merge of two, three or four streams on one channel. The split raised that to 38/40: the trigger fired 85 splits in 40 runs (none to three per run), every target labelled on target, and brought the median time to bind forward (X: 21,600 against 27,600; L: 18,600 against 32,400; medians over bound runs). The two failures (X's seeds 555 and 556) ended with two streams sharing a channel. The reset-only control matched the window gate without the split seed for seed on L (3/5 and 3/5, 0 vs 0 discordant) and bound 0/5 against 1/5 on X: on these seeds the row copy, not the Adam reset, made the difference (Section 6). At four streams, WIN3_SLOW16's two failures were one unrouted bind (X) and one merge (L); WIN16_A's thirteen were four position splits, three merges, four unrouted binds and two others. At two streams no arm failed by position; WIN3's failures were key splits (12 of 15), WIN3_SLOW's key splits and partial streams (6 and 5 of 11), and HINGE0's key splits (10 of 13), with two OTHER and one partial. Slow memory on the window gate at two streams is printed, not claimed: 69/80 against 65/80 (7 vs 3, $p = 0.17$).

What is and is not established. The result is a pre-registered, two-machine, disjoint-seed test of a recipe that uses no stream labels, no restarts and no hinge, and that finds an eight-way partition of the memory, the streams on disjoint channel sets with one channel per stream by largest mass, in 38 of 40 runs where the previous recipe found none. The split needs to know where the key positions are (as the hinge did); the window gate needs the cue within two tokens; and the task is the grouped layout, where it is. Neither limitation is new. The audit's over-prediction for two streams ("$\geq$ 36/40 with KEY failures gone") was not met on X (33/40) and KEY failures were reduced, not removed; what was claimed, G3, holds.

Routing by overlap, intervals, and what a replication should expect. The referee's recount (Section 15) scored the headline runs a second way, by the overlap of the streams' channel sets rather than by each stream's largest channel: in all 38 bound runs the sets are disjoint (largest pairwise overlap 0.047), and under that criterion G4 is unchanged and G1 strengthens to 16 vs 2. The paired differences carry their uncertainty: G1's four-stream difference is 13 vs 2 ($p = 0.0037$), which is secondary, is not shown on X alone (5 vs 1), and would not survive a family-wise correction over the project's 53 claims (Bonferroni 0.20), though it survives Holm within the pooled family of six; G3's two-stream difference of two pairs in the window recipe's favour is $+0.025$ with a 95% interval of $[-0.079, +0.121]$, so "equal at two streams" means within eight points either way, not equal. The eight-stream rate, 38/40, has Wilson interval $[0.835, 0.986]$; a 20-seed replication of a recipe that truly binds 95% of the time reaches ≥ 18 with probability 0.92 and ≥ 17 with 0.98, and one of a recipe that binds 90% reaches ≥ 18 with only 0.68. Section 14 reports a replication at 17/20.

6 The split: the copy, the reset and the trigger

The split bundles two actions, a row copy and an optimizer-state reset, and Revision 7's screens had no reset-only control. Revision 8 reported three looks (Table 3, first three rows), each comparing the copy with its reset against the reset alone; they showed that the reset alone is not enough, and the first draft said more than that. Two further looks settle the question, one of them pre-registered.

where	configuration	copy + reset	copy only	reset only	none	paired
batch 18, S50 (E)	S=8, k=16, 260–269; the reset replayed at the recorded split updates	9/10	—	2/10	3/10	8 vs 1, two-sided $p = 0.039$
session H, S58b	S=8, k=16, 460–469; live trigger	10/10	—	2/10	—	8 vs 0, one-sided $p = 0.0039$
`window_gate` control	S=8, k=16, 540–544, 1540–1544; live trigger	—	—	3/10	4/10	0 vs 1
session H, S61	S=8, k=16, 480–489; live trigger	9/10	9/10; with noise 9/10	4/10	3/10	copy only vs reset only 5 vs 0, $p = 0.031$; vs copy + reset 0 vs 0
`split_copy`, X	S=8, k=16, 560–569; live trigger; pre-registered	**10/10**	—	**3/10**	5/10	C1 7 vs 0, $p = 0.0078$ SHOWN; C2 reset vs none 0 vs 2, HOLDS

Table 3: The split's two actions, bound routed. In every look the reset-only arm fired under the live rule or at the recorded split updates, on targets labelled on target where labelling was recorded, and bound no more often than the arm with no trigger where one exists. S61's copy-only arm is the exact copy of session H (no noise, no reset); its pre-fixed readings "the copy alone suffices" and "the reset adds nothing" are one-pair bounds that both held, and its "the noise matters" reading (≥ 4 vs 0) did not apply (0 vs 0). `test_split_copy`'s L half is running; its pooled claims and readings wait for it.

The pre-registered look (`test_split_copy`, X). Specified (`specs/test_split_copy.md`) before it was written, with three claims pooled primary and per machine secondary. On X (seeds 560–569, 54 runs, every CHECK passed, 14.7 h): C1, the split beats the reset alone, SHOWN, 10/10 against 3/10, 7 vs 0, $p = 0.0078$; C2, the reset alone is not better than no trigger by more than two pairs, HOLDS, 3/10 against 5/10, 0 vs 2 (every seed the reset bound also bound without it); C3 is Section 11's. All 22 splits and all 27 resets were on correctly labelled targets, so the difference is the row copy, not the aim. The validity arms bound 2/2 (the Hebbian perfect gate at 4800 and 3600;

the delta perfect gate at 1200 and 1200). The pooled claims and the pre-fixed readings ("the copy, not the reset" if C1 is SHOWN pooled and C2 HOLDS) wait for L's ten seeds; per machine the reading is met on X.

The copy alone (session H, S61). With the exact copy as its own arm, the three forms of the split bound the same nine seeds and missed the same one (483, two streams sharing): the copy with noise and reset 9/10, the copy with noise 9/10, the exact copy 9/10; the reset alone 4/10, no trigger 3/10. All 83 firings across the four trigger arms copied a channel holding at least two streams onto an empty one. Two identical rows, their symmetry broken only by their different Adam moments and the data, separate two merged streams as well as two noisy ones. The exact copy is therefore the split's form in every later screen and in the next main-line test; the pre-registered test of the copy alone against the full split is to follow.

The constants (session H, S79). The split's constants had never been varied. With the exact copy as the base (9/10, 21 firings, all on target) and one change at a time on ten seeds: a cap of one split, 5/10 (BASE only 4, CAP1 only 0; "sensitive"); checks every 1200 from 2400, 7/10 (2 vs 0; "inconclusive"); a cap of six, 9/10; a 16-sequence probe, 10/10; a rise threshold of 0.05, 10/10 (all "robust"). The pre-fixed reading "the extra splits are harmless" did not apply, because one of the cap-of-six arm's seven extra firings was not on a labelled target (the run bound anyway). Descriptively, and not pre-registered: four of CAP1's five failures spent their single split at updates 4800–12,000 while the probe read 0.09–0.50, before any stream had bound, and the fast-check arm's three failures had used all three splits by 7200–10,800. The plateau rule fires during the flat start, and a split spent there is one fewer for the merge that comes later; the fast check binds sooner when it binds (median 14,400 against 24,000) and fails that way when it fails. What a replication must state is therefore the cap, the first eligible check and the check interval; the probe size and the rise threshold can be stated loosely. Two trigger variants that follow from this were screened on ten fresh seeds (session H's S84, seeds 2300–2309; the oracle 2/2). A guard that forbids splits while the probe is below 0.5 binds 9/10 against the base's 9/10 (0 vs 0); it fires its 17 splits at probe accuracies of 0.50–0.91, where the base fired 7 of its 24 at 0.06–0.16, and it locks out the one seed whose probe never reaches 0.5, which ends MERGED in every guarded arm. Checks every 1200 with the guard bind 9/10 (S79's unguarded fast check, on other seeds, bound 7/10) with a median transition 1200 earlier than the base's, not the 2400 the pre-fixed reading "faster without the cost" required. A trigger that reads the 2048-query held-out evaluation instead of the probe binds 10/10 (1 vs 0; the reading "the evaluation is the better probe" applies by its rule, on a single discordant seed, where the probe read 0.000 then 0.047 and the evaluation 0.066 then 0.068, so the probe's noise decided whether the first check fired). These seeds hardly tested the flat start, since the base kept enough splits for its merges after spending some; the guard changes when the splits fire and, on these seeds, not whether the run binds. Session F's S81 is the same screen on delta channels.

The probe can be wrong about a plateau. The replication of Section 14 lost one of its three failures to the probe: on its seed 914 the 64-query probe read 0.953–0.984 at every check from 16,800 on while the 2048-query evaluation sat at 0.871–0.887 with two streams merged, so the rule never fired again. A 64-query probe near 0.9 has a standard error of about 0.04. The referee then found that the main line's own fixed probe is unbalanced across streams (5, 5, 6, 8, 10, 12, 5 and 13 queries for streams 0–7) and identical for every seed, so that a merge of two under-sampled streams costs the probe about 0.08 where the balanced evaluation loses 0.125: the direction that blocks a split, and a systematic bias rather than noise. How often the main line's checks were blocked this

way cannot be read from its records, which store neither the probe's reading nor the per-stream masses at each check; the next main-line test's records will. The trigger arms above are the remedy under test.

What does not replace it (session H). Two devices the literature uses against router collapse were run beside the split and the reset at four streams, $k = 16$ (40 paired seeds), where every arm but one sat at the ceiling (NONE 38/40) and no reading could apply; a rule fixed beforehand then carried the two best-scoring arms, which were the reset and write-gate noise, to eight streams (10 seeds). Gumbel$(0, 1)$ noise on the gate's logits during training, Raven's device for a router whose reads are dense [13]: on both the read and write gates it collapsed routing even at four streams (33 of 40 runs merged); on the write gate alone it bound 37/40 at four streams with a later transition (median 6000 against 3600) and a lower margin, and at eight streams it bound 0/10, with up to six streams sharing a channel (10 vs 0 against the split; the screen had no no-device arm there). A Switch-style balance loss, as Mixture-of-Memories uses [12, 17]: 36/40 at four streams, routed by stream, but in 34 of those 36 runs the streams were mapped to different channels at key positions than at value positions and the routing margin fell from 0.87 to 0.46; it changed the routing's shape, not its count, and was not carried to eight streams. In every other arm the query's read gate follows the write map (session H's S62: in 32–36 of the bound runs per arm, and 12/12 at eight streams). At two streams ($k = 2$, no convolution, ten seeds), a learned gate temperature with a floor ran to the floor in every run, and a stable-max softmax changed nothing. Slope annealing of the gate (S63) was neutral (9/10 against 10/10), and a hard, one-hot write gate from update 2400 destroyed routing (0/10; KEY 3, MERGED 4, OTHER 3): with the write one-hot, the gate gets gradient only through the read, and the stream labelling of the writes never forms.

The split needs an idle channel. With no spare channel (batch 18, S51, $S = k = 4$), the trigger once fired on a correctly routed map, found no empty channel, overwrote a live one and merged the run. Session H's S78 then ran the exact copy at $k = S = 2$ (Section 9): the copy resets the gate to uniform and the same key split re-forms within 2400 updates. The copy works at eight streams because the idle channel it copies onto is empty; where every channel is busy it has nothing to offer. Spare channels remain the default, and the trigger should not fire when its target already carries key mass.

7 Muon is not an ingredient

Revision 7 reported, from one machine, that under Muon the recipe "binds four streams sooner". The Muon-recipe test (`test_muon_recipe`, the first on disjoint seeds) asked three questions. M1, the early hinge window under Muon at two streams: WIN_M 79/80 against SLOW_M 69/80 pooled, 10 vs 0, $p = 0.001$, SHOWN (on L 6 vs 0, SHOWN; on X 4 vs 0, $p = 0.0625$). M2, Muon binds four streams by update 4800 more often than Adam: 12 vs 8 pooled, $p = 0.25$, NOT SHOWN; per machine it was shown on X (7 vs 1, $p = 0.035$) and reversed on L (5 vs 7). Revision 7's result had come from L (7 vs 0 on its seeds; X's cut arm gave 5 vs 1), so on fresh seeds it was confirmed on the other machine and reversed on its own; the machines disagree, by seed block or by CPU (the design cannot separate them), and pooled it is not shown. M3, the same at eight keys: 7 vs 5 over 30 pairs (X's time rule cut its Adam arm to ten seeds), $p = 0.39$, NOT SHOWN. The test's reading, "the Muon recipe is reliable at the working configurations", was not met: WIN16_M was MAJORITY pooled (33/40). Batch 18 then ran the window recipe with the split under Muon at eight streams: 9/10, the same as under Adam (1 vs 1), and 9/10 against a fresh Muon reference

without the split at 2/10 (7 vs 0, two-sided $p = 0.016$). The eight-stream result is the recipe's, not the optimizer's. Muon's bf16 Newton–Schulz step does not reproduce across processors, which cost the project paired comparisons; nothing in the main line now needs it.

8 The recurrent gate's eight-stream failure, closed

Revision 7's reading was that the recurrent gate's state stops carrying the stream when its recurrence becomes expansive, and it left a cap on that recurrence being screened. Batch 15 (S41) ran the cap at 0.5 on the spectral norm of W_h under Muon, alone, with a previous-token input, and with the previous-token input and no recurrence, and the capped previous-token input under Adam: the three Muon arms bound 0/10 and the Adam arm 0/5. With the look-back the capped gate's state kept the stream at key positions throughout (decodability 0.93–1.00) and still ended with the eight streams on three to seven channels; final accuracy rose in every pair (median 0.61 against 0.36) but no run bound. Batch 16 added a frozen reservoir recurrence (0/10), decay 0.98, a tenth of the rate on W_h, and a higher learning rate: none bound (the perfect gate binds at decay 0.98, so the decay is valid). The window gate then bound where all of these had not. Two corrections to Revision 7 follow: the gate's state must carry the stream at the *write* position, where CTX is two tokens back, not only at the read, where it is one back, so the sentence "the gate needs only one token of memory" was wrong (the audit noted it; batch 16's S47 measured the write positions, which S38–S41 had not); and S41's no-recurrence arm, whose input is (v_t, v_{t-1}), cannot by construction see the stream token at value positions, so its ten key splits were a property of that window and not evidence about recurrence (audit, Section 2.3). The cap, as specified, also acted from update 1 ($\sigma_{\max}(W_h) \approx 1.08$ at initialization), not only after the crossing. The diagnosis stands as a description of the recurrent gate; the remedy was to remove the recurrence.

9 Two-stream key splits

At two streams with two channels and no convolution, the window gate fails by routing on key identity: its key splits form within the first 600 updates, while the memory is still at 10^{-4} (batch 17, S49). Slow memory reduces them (WIN3 KEY 12 of 15 failures pooled; WIN3_SLOW KEY 6 of 11), and batch 18 screened two further devices on seeds 160–199: two spare channels ($k = 4$: 36/40 bound routed, with four key splits still formed by update 325) and the hinge's key term on updates 1–2400 (36/40 discovered, one KEY failure, but three "STREAM-PARTIAL" failures at $\eta^2 = 0.5$ by key, two at 0.5 and one at 0.4999, the signature of a hard two-of-four key pattern that the key-first classifier labels KEY). The configuration is the project's hardest relative to its size and the one with the least headroom (65–69/80 for every arm); it is also the only one where the window recipe's margin over the hinge recipe is small (two pairs pooled, $[-0.079, +0.121]$) and where the hinge recipe won a screen (batch 16: 34/40 against 31/40 on seeds 160–199).

The split does not repair a key split (session H, S78). Session H ran the exact copy at $k = S = 2$ on 20 seeds (1300–1319), with the main line's noisy form and the two-spare-channel recipe beside it: the baseline discovered 15/20, the exact copy 16/20, the copy with noise and reset 17/20, four channels 15/20; no arm beats the baseline (best 2 vs 0, $p = 0.25$), and the pre-fixed reading is "neither device". The firing records say why. After $W_g[c_0] := W_g[c^*]$ the two rows are equal and the gate is uniform, every η^2 at 0.00; by the next check, 2400 updates later, the same key split had re-formed exactly (KEY/stream η^2 1.00/0.00) after all eight firings on the three seeds that ended KEY, and the half-key pattern after all three on a fourth. The one rescue was a seed whose

gate was not yet a key split when the copy came. With noise and the Adam reset the key split re-formed only partly on two seeds and fully on one. At eight streams the copy creates a near-tie between a crowded channel and an empty one; at $k = S$ it only resets the gate to where the gradient takes it again. Session H's reading of the mechanism, that W_g's untouched Adam moments still point along the old split, was not measured and is not a reading.

What is left. Prevention rather than repair: a gate that cannot commit to a key split while the memory is slow (the gate group at 10^{-4} for the first 1200 updates), and the key term of the hinge under the key-first classifier, are running on 20 seeds in session H's S85. Under the classifier of record, four of S78's seventeen unbound runs change class from STREAM-PARTIAL to KEY; no reading changes.

10 Scale

Batch 19 (E, running) asked whether the recipe's constants are tuned to $N = 256$, $D = 32$, three layers. At four streams with sixteen channels (seeds 270–289, the oracle binding at every size and depth), with every constant unchanged: $N = 512$ and 1024 at $D = 32$, and $D = 64$ and 128 at $N = 256$, bound routed 10/10 each against the reference's 9/10, all with median transition 3600; 2, 4 and 6 applications of the shared layer bound 10/10, 10/10 and 9/10 against 10/10 for three, with six layers slower (median 4800). Both readings were "scale-free" by their pre-fixed rule (every $d \geq -1$), at four streams only; with ten seeds the screen rules out a large drop, not a small one. Eight streams and larger key loads at the larger widths are running.

11 The memory rule

What the literature holds. Twelve papers on routing, memory and context were read in full (`docs/reading/`): the test-time-regression framework [5], Gated DeltaNet [6], the Tiny Recursive Model and recurrent-depth language models [7, 8], Flesch et al.'s Hebbian context gating [9], Active Dendrites [10], Context-Gated Associative Retrieval [11], Mixture-of-Memories [12], Raven [13], a study of content-based routing [14], the Hierarchical Multiscale RNN [15] and Nested Learning [16]. None of the twelve carries an inferred context cue across distance without labels under a sparse end-of-sequence loss (the Hierarchical Multiscale RNN infers its boundaries from a dense likelihood, as our note records): each hands the context over (a task identifier, a prototype readable off every input, a clamped context layer), carries it with softmax attention, or routes by token type. The study of content-based routing is the sharpest statement of why our header-layout runs failed: a router over a frozen state-space model's summaries reaches about 29% of the right positions even with the route supervised, over Fourier-mixed representations 0.9%, and 98–99.7% once an upstream, jointly trained attention or mixing layer has written the context into each token's representation. Flesch et al. fit a leaky integrator of the context cue to human learners and show that slow integration of interleaved contexts merges them, and that inside a block the loss exerts no pressure to use a constant cue. Our window gate reads three raw embeddings; our fading-memory gates pooled everything; our fixed header layout is positionally solvable. Those are the failures the papers' analyses, read as analogies, predict.

BDH-CQ. Pathway's follow-up [2] describes its memory as "linear correction rules on S" with linear attention as the special case $S_t = S_{t-1} + U_\theta(D_t)$, and its reasoning as $H_{r+1} = F_\theta(H_r, S_K)$ over a frozen memory; "dimensions, exact update rules, and implementation details remain proprietary".

Our reading (`docs/reading/`, an inference from the sentences quoted above, not a finding, and one reading among several, since the quoted sentence counts plain linear attention, which does not read S, among such rules): in the taxonomy of [5] and [16], the rules that are affine in S and read S before writing are the delta-rule family (DeltaNet, Longhorn, Gated DeltaNet, DeltaProduct), and the loop matches the recursive-refinement models [7, 8]; the recurrent-depth model re-injects its input embedding at every step to keep the recurrence stable, and the frozen memory S_K in BDH-CQ's loop would be the analogue, which is our analogy and not theirs. Neither is a distant-cue mechanism. The 97.4% Sudoku figure sometimes attached to BDH is from Pathway's blog, on internal data with an unstated protocol, and is not comparable to the Sudoku-Extreme numbers of [7].

The premise: one channel cannot. One Hebbian channel binds the two-stream grouped layout 0/60 without the convolution and a minority with it at a higher learning rate (18/40, 10/20), and four streams 0/20 (Phase IV). Session F's delta-rule channel (Section 3) was the stronger single memory the project lacked: at a fixed decay it binds the two-stream *header* layout 8/10 (S53, $P = 4$) and the grouped two-stream layout 4/10. The premise test then ran it where the partition's claim lives (S68, seeds 300–309, 43,200 updates): one delta channel binds the grouped layout 2/10 at four streams and 0/10 at eight, while both delta perfect gates bind at 1200 and the gated Hebbian recipe binds 19/20 and 9/10 on other seeds; the pre-fixed readings "context-tagged keys scale" ($\geq 7/10$ at eight) and "the partition's advantage is in the number of streams" ($\geq 7/10$ at four) both read "neither". The main line's pre-registered version, `test_split_copy`'s C3, is SHOWN on X: the recipe 10/10 against one delta channel 0/10 (10 vs 0, $p = 0.00098$; final accuracy 0.08–0.76), one Hebbian channel 0/10, the delta perfect gate 2/2 at 1200, so that the channel's failure is the missing partition and not the delta rule; the pooled claim and the reading "the partition is necessary at eight streams against a single delta channel" wait for L.

The forget gate needs the delta rule. On the fixed eight-key header layout a single delta channel with a learned forget gate (Raven's routed decay, or Gated DeltaNet's) bound 9/10 and 8/10 against 3/10 at the fixed decay (S59), and, post hoc, had learned to partly clear its memory at the context token (median α of 0.18–0.26 at CTX in the first layer, 0.72–0.74 in the third, 1.00 at keys and values). The same decays on a single *Hebbian* channel, same seeds and batches, bound 0/10, 0/10 and 0/10 (S69): the Hebbian channel does not learn the clearing (median α at CTX 0.975 and 0.988 in the first layer), and the learned decay lowered its final accuracy (0.17 against 0.22).

The delta recipe keeps eight streams (session F, S76). Revision 8 reported that the window-gate recipe on delta channels stalled at eight streams (3/10 against 9/10, S54(c)) with a perfect gate that itself took 22,800–28,800 updates, and concluded that the Hebbian write stays on the main line. S76 repeated the comparison with the write strength fixed at $\beta = 1$ and the exact-copy split. Its first phase ran the perfect gate on four seeds: with L2-normalised keys the delta oracle binds all four at the first evaluation (1200) at $\beta = 1$ and at 2400 at $\beta = 0.25$, sooner than the Hebbian oracle (2400–6000); with raw keys one seed diverged and one bound late (16,800). The pre-fixed readings for the slowness (the overwrite; the key normalisation) both read "neither", because no L2 arm was slow. Its second phase ran the recipe, paired, on ten seeds: on delta channels 9/10 bound routed against 10/10 on Hebbian channels (1 vs 0), sooner on eight of the nine seeds both bind (median 14,400 against 24,000); the reading "the delta recipe keeps eight streams" applies. Six of the twelve splits on delta channels fired before the map had merged (their targets not labelled on

target), mostly as single splits in runs that then bound: the plateau rule is tuned to the Hebbian trajectory. S54(c) differed from S76 in two things, the learned β and the noisy split with a reset; the oracle comparison holds the routing fixed, which makes the learned β the likely cause of the earlier stall, consistent with its collapse toward zero in S53 and S54(d), but the screen did not separate the two and that attribution is post hoc.

. . . and two and four streams (S80). The same recipe on delta channels, paired with Hebbian on ten seeds each: at two streams ($k = 2$, no convolution) 10/10 against 10/10, delta at the first evaluation on every seed against 2400–6000; at four streams ($k = 16$, with the convolution) 10/10 against 10/10, both at 3600 on every seed, no split firing in either arm. Both readings are "keeps it". Both arms sit at the ceiling, so ten seeds can only detect a large loss: at the main line's rates (0.86 and 0.95) a Hebbian arm loses about 1.4 and 0.5 runs in ten, and "keeps it" means that no seed failed on delta, not equivalence at the main line's power.

What the delta stack carries. Session F's one-seed observation in Revision 8, keys at the third layer tagged by stream, did not generalise. Reruns of 30 recorded single-channel runs with a linear probe at the end (S77; every curve bit for bit) found the stream linearly readable from the residual at layer 3 in one of sixteen bound delta runs on the fixed header layouts (medians 0.57–0.67), and the two S53 runs that did carry it at value positions (0.92–1.00) were the two that failed to bind; the Hebbian channel carries nothing anywhere (0.50–0.53). Keys can be near-orthogonal in the 256-dimensional code without a stream direction in the residual. On the grouped two-stream layout (S80) the delta stack does carry the stream at *key* positions, from layer 2 on (1.00 at layer 3 on all ten runs; Hebbian 0.57), and not at value positions (0.54–0.65), in the perfect-gate arm as well; session F reads this, post hoc, as adjacency to the context token that precedes each key rather than a tag carried to the write. On the randomised header layout (Section 12) the stack carries the stream only in the solutions that bind without routing.

What this changes. "The Hebbian write stays on the main line" is withdrawn. With the write strength fixed, the delta write is at parity with the Hebbian write at two, four and eight streams on screens, binds sooner at two and eight, binds the randomised header layout 14/20 against 1/20 for the window baseline (Section 12), tolerates an unrouted first layer (S71), and still needs the partition at four and eight streams (S68). The question is a main-line one again, and `test_delta_recipe` (specified in `specs/`; not run) puts it: the exact-copy recipe on delta and on Hebbian channels, 20 seeds per machine, with the claims that delta is not worse than Hebbian by more than two pairs (D1, a bound), that the split is an ingredient on delta channels (D2) and, secondary, that delta binds sooner (D3); its reading "the memory rule is open" requires D1 to hold and D2 to be shown pooled. Its specification is the first to go to the referee before any run. Raw keys are excluded by their numerical fragility ($\beta\|x\|^2 > 2$ makes $I - \beta xx^\top$ expanding), and the trigger on delta channels is the subject of F's S81.

12 Distant cues

The window gate sees three tokens, so on the header layout it cannot see the cue from the third pair of a block on. Revision 8 left the distant-cue problem open with three proposals from the audit: probe where the cue is in the residual stream, add a register that overwrites on the cue and holds otherwise, and let a per-layer gate read the residual stream. Session G ran all three, in two rounds, and the second round changed the answer.

The cue is not in the stack (S55). A logistic-regression probe from the residual stream entering each layer to the stream, at key and value positions, on the trained window-gate stacks of the eight-key header layout: 0.55–0.58 at layers 2 and 3 and at the output, from about 0.70 at initialisation, where the decayed sum of the block still carries the header. Training removes the cue: the gate reads only the window, nothing asks the stack to keep it, and the perfect gate's stack carries nothing either (0.54). The pre-fixed reading "absent" applied, and a gate over the residual stream therefore had nothing to read, which the routing study predicts [14].

The register at layers 2–3 fails on Hebbian memory, and the reason was layer 1 (S56, S57, S71). With the register gate at layers 2 and 3 and the window gate at layer 1, on the fixed eight-key header layout, every Hebbian arm bound 0/10: the baseline, the residual gate alone, the register with and without a stop-gradient, and the register with a 5% labelled nudge. The nudged arm is the informative failure: its register writes a clean latch (β^R 0.96 at the context token against ≤ 0.05 at keys and values), its gates route one-to-one with margin 1.00 at layers 2 and 3, and its accuracy plateaus at 0.32 on *both* streams in every run, no better than the unrouted baseline (0.28–0.42). A perfect gate at layers 2–3 alone bound 0/3 where the full perfect gate bound one of two seeds, one paired seed of evidence. On delta channels the same architecture binds: the nudged register 8/10, routed 7/10, transitions 3600–6000; the unnudged register 6/10 bound against the baseline's 2/10 (4 vs 0, $p = 0.0625$), but only two of its six routed, three binding through key splits. The Hebbian perfect gate itself is marginal on this layout at 24,000 updates (3 of 6 seeds over the three screens), so those zeros are weak. Session G then put the layer question on ten paired seeds of the randomised header layout, whose perfect gates bind at 1200 (S71): on Hebbian memory the perfect gate at all three layers binds 10/10, at layers 2–3 alone 0/10, every run at exactly chance between the two streams (0.46–0.51 in every (stream, block, pair) cell) with layers 2–3 routed perfectly, and at layer 1 alone 9/10, slowly (median 16,800); on delta memory the perfect gate at layers 2–3 alone binds 10/10. The pre-fixed readings "layer 1 is the block" (10 vs 0, $p = 0.00098$), "layer 1 alone suffices" and "delta tolerates an unrouted layer 1" all apply. The interpretation, which is ours and not a reading, is that an unrouted Hebbian memory at the first layer mixes the streams before the routed layers read it; Revision 8's design kept layer 1 on the window gate, and that choice is what sank its Hebbian register.

The register at all three layers carries the cue; discovery is the gap (S72, S73). One register gate shared by all three layers (REG3, Section 3), on the randomised header layout, 20 seeds, readings on ROUTED-BOUND. With the 5% labelled nudge it carries the cue every time: **20/20** on Hebbian memory (transitions 2400–8400) and **10/10** on delta (all at 1200), one-to-one with margin 1.00 at every layer. Without the nudge it finds the solution on 4/20 seeds on each memory, against 0/20 for either baseline, each discovering run routed perfectly; both screens are one discordant pair short of their $p < 0.05$ rule (4 vs 0, $p = 0.0625$), so the pre-fixed reading is "not discovered" on each, and 8 vs 0 over the 40 pairs is a description, not a test, since the two screens share seeds but not models. The S73 reading was fixed on ROUTED-BOUND; on the outcome named DISCOVERED the same arm is 5 vs 0 ($p = 0.031$), one run binding and separating its values with a margin of 0.66, which is why Section 3 fixes the names. Descriptively: in every routed unnudged run β^R at the context token rose above its value at keys and values (Hebbian 0.32–0.64 against 0.16–0.24; delta 0.36–0.58 against about 0.2), and in the runs that failed the gate had locked into a key split (KEY 14 of 20 on each memory by class) with β^R flat across roles. The unbound Hebbian runs sit at chance between the streams in every cell, as the layers-2–3 oracle did. The other devices of the screen: the residual gate alone routes 0/10 on delta (5/10 bound, three by key splits);

the HM-RNN latch with slope annealing holds an exact latch when nudged (9/10 routed, $z = 1$ at the context token and 0 elsewhere) and, unnudged, goes to $z = 1$ everywhere by update 1200 and key-splits 9/10, as it had on the fixed layout; one delta channel binds the layout 7/10 with no gate. The in-run probe finds the stream in the residual of no routed run and of most non-routing solutions (the single channel's bound runs at 0.93–1.00; the half-routed baseline's at 0.86–1.00): on delta memory the stack can learn stream-tagged keys, and when it does the gate does not need to route, which is a description of a 2×2 the screen fixed as descriptive.

Holding the key split off lets the write appear (S82). The discovering runs fail by a key split that forms first (Section 9), so session G screened three devices against the key split on 20 fresh seeds (2100–2119), Hebbian memory, with the baseline, REG3 and a nudged pair for validity (2/2, with the perfect gate 2/2). The hinge's key term of batch 18, a penalty on the shared gate's dependence on key identity at key positions during updates 1–2400 (REG3_KEYHINGE), binds routed **20/20** against the baseline's 0/20 and REG3's 2/20 on these seeds (20 vs 0 and 18 vs 0; the readings "discovered" and "helps" both apply, and the arm has no key failures and β^R at the cue above keys and values in every run); on delta memory the same arm binds 10/10 against 0/10 and 2/10. A gate held slow for the first 1200 updates binds 0/20, worse than nothing, because it delays the register's write along with the split; a gate without the raw window input binds 5/20, "discovered" exactly at its threshold (5 vs 0), with a register that writes at nearly every token. What the firing records show, as a description: the near-uniform gate's small variations are organised by key from initialisation (η^2 by key 0.89 at update 1), the hinge holds that at 0.05 from update 300, the register's write at the cue separates from its write at keys and values during the hinged phase (0.12 at every role at 1200; 0.48 against 0.20 at the end), 18 of the 20 bind at 3600, the first evaluation after the hinge stops, and the key split does not return once the gate routes. The hinge reads the task's key positions and key identities, as the split reads key positions: it is a training-time prior over the first 2400 updates and not an architecture, and in a language model key identity would be token identity, which a gate must in general be allowed to use.

What this changes. The distant-cue task now has a solution without stream labels: a register that overwrites at the cue, a gate over the register, the window and the residual at all three layers, and a hinge that keeps the gate off key identity while the memory is slow. Its capacity was never the limit (the nudged register, 20/20); its discovery is 20/20 with the hinge and 2–4/20 without. Session G proposed the pre-registered test, and it is the register test queued behind `test_delta_recipe`: the hinged register against the baseline, against the register without the hinge, and against the hinge on the window gate without the register (a control the screen did not run), 20 seeds per machine, with a rate claim of $\geq 18/20$ on each. The audit's per-layer gate over the residual stream is closed: with nothing in the stack to read, it key-splits (0/10 twice).

13 Where the mechanism stands

setting	perfect gate	recurrent gate, Rev. 7 recipe	window gate + SLOW	window gate + SLOW + split	the same on delta channels[d]	one Hebbian channel	one delta channel
S=2, P=4, no conv.	4/4	67/80	69/80	—	10/10 vs 10/10	0/60[c]	4/10[f]
S=4, P=4, k=16, conv.	4/4	27/40[w]	38/40	39/40[e]	10/10 vs 10/10	0/20	2/10[f]
S=8, P=4, k=16, conv.	4/4	0/40	15/40	**38/40**; X 10/10[s]	9/10 vs 10/10	0/10 (X)	0/10 (X), 0/10[f]

Table 4: The grouped layout: the main configurations of `test_window_gate` (pooled over X and L, disjoint seeds; bound routed, or discovered at $S = 2$), with `test_split_copy`'s X half and the screens. [w]With the early hinge window (WIN16_A); Revision 7's full-hinge arm bound 35/40 on shared seeds, routed 15/20 on X and 16/20 on L. [e]Session H's S58, exploratory, 40 seeds. [c]One Hebbian channel without the convolution, every such run on the grouped layout in `results/` (the blocked and shuffled layouts add 0/30); with the convolution at learning rate 4×10^{-3} it binds 18/40 and 10/20 at two streams (Phase IV's `conv_lr`; Revision 6 reported 34/80) and 0/20 at four (`test_scale_axes`, B4s). [d]Session F's S80 and S76, exploratory, ten paired seeds each, the delta recipe against the Hebbian one with $\beta = 1$, L2 keys and the exact-copy split. [f]Session F's S53 and S68, exploratory, $\beta = 1$ at the fixed decay. [s]`test_split_copy` on X.

randomised header layout (S=2, P=4, k=2, 24,000; session G, seeds 700–719)	Hebbian memory	delta memory
perfect gate, all three layers	10/10	10/10
perfect gate, layers 2–3 only (window gate at layer 1)	0/10	10/10
perfect gate, layer 1 only	9/10	—
window gate + SLOW (ROUTED-BOUND; bound)	0/20 (1/20)	0/20 (14/20)
register gate at all three layers, labelled nudge	**20/20**	**10/10**
register gate at all three layers, label-free	4/20	4/20
the same with the key hinge for updates 1–2400 (S82, seeds 2100–2119; baseline there 0/20, register 2/20)	**20/20**	**10/10**
the same with a slow gate for 1200 updates / without the window input (S82)	0/20 / 5/20	—
residual gate at all three layers, no register	—	0/10
HM-RNN latch, label-free / nudged	—	0/10 / 9/10
one channel, no gate (bound)	—	7/10

Table 5: The distant-cue task. Exploratory (session G, S71–S73 and S82); the perfect-gate rows are BOUND, the others ROUTED-BOUND, and session F's window baseline on other seeds (S70, 330–349) is 0/20 routed on either memory with 1/20 and 6/20 bound.

The partition does what one channel cannot, and the gate finds it where the cue is local. Given the partition, every grouped-layout configuration tried with the $N = 256$ model binds, on either write rule. One Hebbian channel does not; one delta channel binds a minority at two streams and none at eight, and the pre-registered version of that comparison is SHOWN on X. A gate that reads three tokens, trained with the memory held slow for 2400 updates and with a channel split when binding stalls, finds the partition at two, four and eight streams without stream labels or restarts, and the split reads the task's key positions.

Discovery is a race, and the recipe's parts each remove one way of losing it. Slow memory reduces the key splits (12 of 15 failures to 6 of 11 at two streams; the count itself, 7 vs 3, is not shown); the window gate cannot count positions, so position splits are gone; the split breaks the merges that remain at eight streams by giving an idle channel the busy channel's logits, and it is the copy that does it. Two ways of losing remain: at two streams a key split that the copy cannot undo, because there is no idle channel; at a distance, a key split that forms before the register's write at the cue has appeared, which a hinge on the gate's key-dependence during the slow phase prevents (20/20 on a screen).

The write rule and the layer. The delta write with a fixed write strength is interchangeable with the Hebbian one on the grouped layout at every size tried, and better placed for distant cues, where a Hebbian stack needs its first layer routed and a delta stack does not. The partition's claim is about the number of contexts on both rules: one delta channel binds the two-stream layouts by routes that need no partition and fails at four and eight streams.

14 An independent replication

Design. A separate session (R) was given Revision 8's PDF and Pathway's public BDH repository and nothing else, with the rule that it must not look at this project's code, branches or web presence until its own records were pushed. It implemented the task, the model, the outcomes and the recipes from the text (Phase 1), logging every detail the report did not determine in a file of 44 entries (`UNSTATED.md`) before running; pre-registered its readings; ran the perfect gate on two seeds and the eight-stream recipe with and without the split on twenty fresh seeds (900–919) at the report's budget (Phase 2); and only then fetched the main line and diffed the two implementations (Phase 3), rerunning ten seeds with one discrepancy at a time set to the main line's choice. Its bar, fixed in advance: "replicated" at $\geq 18/20$ bound routed with the split beating the no-split arm at $p < 0.05$; "partially replicated" at $\geq 12/20$.

Result. The perfect gate bound 2/2. The recipe with the split bound routed **17/20** (Wilson $[0.640, 0.948]$); without it 7/20; every discordant pair favoured the split (10 vs 0, $p = 0.00098$); all 45 of its splits but one copied a channel holding two or more streams onto an empty one; every failure in both arms was a merge of two or three streams, as in ours. The mechanical reading is "partially replicated", one run short of the bar. Against the report's 38/40 the count is Fisher two-sided $p = 0.32$, and pooled over X, L and R the rate is 55/60 $[0.819, 0.964]$; the referee's assessment (Section 15) is that the bar could not separate the outcomes it was meant to (it fails a true 0.90 recipe a third of the time) and that the defensible statement is R's own: 17/20 is consistent with 38/40, and twenty seeds cannot tell 0.85 from 0.95. The two reruns with the main line's probe and with its key positions left the count unchanged on seeds 900–909 (9/10, 9/10), and were not run on the one failure a protocol difference plausibly explains (914, the probe reading of Section 6). A fourth phase, with the recipe as the main line specifies it and a reading calibrated to its size, is running.

What the report determined, and what it did not. R loaded its weights into our model: the gates agree to 7×10^{-9} and the query logits to 7×10^{-8}. The architecture, the gate, the convolution, the initialisations, the SLOW groups and switch, the batch, the evaluation and the outcome definition were all determined by the text, and R found nothing in our code that contradicts the report and nothing it had misread. Every difference is protocol, and five could change an

outcome: the identity of the split's probe (ours is one fixed set for every seed and arm; R drew one per run), the key positions whose gate mass picks the channels (ours the body's 32 keys; R's included the query's), the perfect gate's schedule (ours a single rate; R's SLOW, which explains its slower oracle), the order of the trigger and the early-stop test at an evaluation step, and the last eligible check. Nine of the 44 guesses were material and all nine matched: decay rather than RoPE; the 0.02 embedding initialisation, which sets the scale of the gate's input; that the gate reads the model's own raw embedding; the window gate's width and initialisation, carried over from the recurrent gate; the convolution as one shared width-4 identity filter on the layer input, feeding keys and values; Adam without weight decay and without dropout; the probe's reference at 2400; the zeroed Adam step counter; the map as the argmax of the mean gate at value positions. Appendix B states all of them, with the three the referee added (the probe's composition, the outcome's boundary rules, a record format an outside scorer can use).

Caveats. Isolation rested on self-report. The PDF R was given lay inside a checkout of this repository in its container, and a filename search printed six of that checkout's filenames; R opened none of them and judged the replication not void, and so do we, but a second replication should be isolated by construction (Section 17). R used PyTorch 2.14.1 (ours 2.14.0), four workers on one CPU model, and 37 hours of compute; bit-for-bit agreement across implementations was neither expected nor sought.

15 The referee's report and the claim registry

What the referee did. A session with no part in the work (V) was given the repository and asked to review Revision 8 adversarially, changing nothing outside its own directory, which is now `docs/review_2026-10/`. It recomputed every count, discordant pair, p-value, interval and band of the report's tables from the JSON records under outcome definitions it re-implemented (140 matching rows; two later passes over the exploratory sessions' reports, 808 rows, found every pre-registered reading re-deriving exactly and 13 disagreements, all in descriptive ranges); audited the git history of every main-line claim (no claim or reading edited after a run; the split arm added by a dated amendment before any learned-gate result existed) and of 21 exploratory screens; reproduced one run of each Phase VI test bit for bit on a third CPU model; checked the literature against the project's own notes; and compiled the claim registry of Section 3. Its recommendation on the first draft was "major revision": the central experiment sound, the manuscript claiming more.

What it found wrong, corrected in Revision 8.1. Seven statements were false when the first draft was committed: that no single-channel run had been made at eight streams (session F's premise test was on its branch 82 minutes before the commit, and supported the reading); the Muon correction with the two machines the wrong way round (Revision 7's result came from L, was confirmed on X's fresh seeds and reversed on L's); an argument about the convolution that compared an $N = 64$ ceiling model with the $N = 256$ model (the $N = 256$ perfect gate without the convolution binds on every seed recorded); batch 16's hardware; a stale count of single-evaluation binds; a claim that every test had a specification file; and Revision 7's routing "recorded only on X" (L's records carry it). Seven statements overstated their evidence and were reworded: "the reset adds nothing" (the looks cited showed only that the reset alone is not enough; Section 6 now has the look that shows it), "one channel per stream" (by largest mass; the sets are disjoint), "no labels" (no stream labels; the split reads key positions), Gumbel noise "on the read gate", "beat" for a

screen whose verdict was "promising", the literature sentence on inferred cues, and the BDH-CQ reconstruction. The referee's check of the corrections found two slips in them (a run count; a phase number) and one older error both had missed (one Hebbian channel *had* been run at four streams: 0/20), corrected in a second pass.

What it asked for, and what this revision does with it. Scope (done: the title and Section 18); the copy-only arm as a pre-registered claim (Section 6; the copy against the reset is pre-registered and shown on X, the copy alone against the full split is next); the split's dependence on key positions (stated everywhere it is claimed; a trigger that reads all positions is not yet screened); an external baseline family (one delta channel and Gated DeltaNet's write are now run at every size; a residual-stream router was run and failed; Mixture-of-Memories' balance loss was run); paired-difference intervals and a stated error rate (Section 5, Section 3); routing by overlap (reported); the literature's remaining overstatements (Section 11); provenance (commits pinned; L's local start times noted; L's Phase III–IV records committed, Appendix A). Two requests are open: the claim list's q-values are in the registry and not yet in the tables here, and BDH-CQ and Triadic Linear Attention have still not been read in full.

The registry's two findings about the records. L's result files for Phases III–IV were not in the repository: the commit that was meant to hold them (5 October) held byte-identical copies of X's files, naming X's processor, so every L verdict from those phases was known only from text written later. The user recovered twelve of the thirteen files from the laptop's working copies and reran the thirteenth; all 90 L rows the registry can now check against a file agree with it (38 verdicts, of which 17 had been second-hand, and the 32 bands, gates and readings recounted from the per-run records), eleven L tests gained a printed p, and L's family under correction improved (27 of 32 positive verdicts survive Benjamini–Hochberg). The main line's split probe is unbalanced across streams (Section 6).

The sessions' reports, after the referee. Statements the sessions made beyond their pre-fixed readings are quoted in this revision as descriptions, not results: among them, that S54's slow delta oracle was a learned-β artefact, that the register's write at the cue is what separates a discovering run, that the stack's stream tag lets the gate stop routing, that noise helps a little at two streams, and that the check interval and the first eligible check act through the flat start. Each is marked where it appears.

16 Retrospective

16.1 Revision 8's claims

"Eight streams bind without labels or restarts" (the main result). Stands, and was replicated from the text alone (17/20; Section 14). Qualified: without *stream* labels; the split reads key positions; "one channel per stream" by largest mass, the sets disjoint.

"The copy, not the reset." Pre-registered and shown on one machine (C1, 7 vs 0; C2 holds), pooled pending L. The first draft's "the reset adds nothing" was not shown by the looks it cited; session H's S61 shows it on ten seeds (the copy alone 9/10, the full split 9/10), and the pre-registered version of that comparison is next.

"The Hebbian write stays on the main line." Withdrawn. With the write strength fixed, the delta write matches the Hebbian one at two, four and eight streams and binds sooner (Section 11); the stall Revision 8 reported came from a screen with a learned write strength. A pre-registered test is specified.

"The partition's claim is about the number of contexts." Supported: one delta channel 2/10 at four streams and 0/10 at eight (S68), 0/10 in the pre-registered test on X (C3).

"If the stream-tagged keys at the third layer hold up, the context is reaching depth." They did not (1 of 16 bound runs; Section 11); the context reaches depth on delta memory only in the solutions that bind without routing.

"The next steps are a probe, a register and a per-layer residual gate." Done. The probe found the cue absent; the residual gate had nothing to read; the register carries the cue once it also gates the first layer, which Revision 8's design did not do, and that design choice is what sank its Hebbian register (Section 12).

"Scale-free" at four streams (batch 19). Stands as a screen; the referee notes that its one-pair rule on ten seeds reads "scale-free" 39% of the time when the larger model's rate is 0.7 against 0.9, and that "depth" is applications of one shared layer. Eight streams at the larger sizes (S66, S67) are running.

16.2 Revision 7's claims

"At eight streams the gate forgets the context" (subtitle). Superseded. True of the recurrent gate; the window gate carries the context at every write position by construction and binds 38/40.

Revision 7's recipe (slow memory and a hinge) binds without restarts at two streams and at four with sixteen channels. Stands, and is replaced by a recipe without the hinge, which is within eight points of it at two streams ($[-0.079, +0.121]$), beats its early-window form at four (G1) and beats it at eight (G2).

"The gate needs only one token of memory" (Section 11 of Revision 7). Corrected. One at the read, two at the write; the write-position decoders were not measured in Revision 7's screens S38–S41 (batch 16, S47, measured them).

"Under Muon the recipe binds four streams sooner." Not shown on disjoint seeds pooled (12 vs 8, $p = 0.25$), and the machines disagreed: confirmed on X's fresh seeds (7 vs 1), reversed on L's (5 vs 7), where Revision 7's result had been shown. Revision 8's first draft called it a seed effect, which the data do not establish; the referee's heterogeneity test between the machines ($p = 0.070$) is the more interesting figure, and a seed-block swap between X and L would separate processor from seeds.

"A cap on the gate's recurrence is being screened." Resolved: not. No capped arm bound (0/10, 0/10, 0/10 and 0/5); the cap acted from update 1.

The row copy breaks merges (Section 10 of Revision 7). Supported with the right outcome and a reset-only control in four exploratory looks and, on X, pre-registered.

16.3 Errors made during Phase VI

Statements that were false when committed. Listed in Section 15. The common cause was writing from a summary of the branches rather than from the branches: the premise test and the copy-only screen had both been pushed before the draft was committed, and neither was checked. The fact-check that preceded the draft was done by the author of the draft.

Predictions that the data overturned. We predicted that a single delta channel could not bind conflicting streams (it binds the two-stream header layout 8/10); that Gumbel noise on the gate's logits could replace the split (on both gates it collapsed routing; on the write gate it bound 0/10 at eight streams); that a balance loss would attack merges (it changed the key- and value-position maps instead); that a learned forget gate would widen the gap between the oracle and one channel (it shrank it, on the eight-key header layout); that the register at layers 2–3 would carry the cue on Hebbian memory (it needed layer 1); that the delta recipe would stay behind at eight streams (it is at parity with the write strength fixed); that a single Hebbian channel's stream tag, if any, would be linearly readable (it is not); that a dense loss would need the partition at two sources (one channel self-tags); that a gate held slow at the start would let the register's write win the race against the key split (it binds 0/20, worse than nothing, because it slows the write too); and, in the audit, that the window gate with slow memory would reach $\geq 36/40$ on seeds 160–199 with no key failures (it reached 31/40 there, and 33 and 36 of 40 on the main test's seeds, with key failures reduced).

A design choice that cost a round. Revision 8's register specification kept the first layer on the window gate on the reasoning that its residual input is the embedding and carries no cue; but the register is built from embeddings and could have fed the first layer's gate, and on Hebbian memory the first layer is the one whose routing matters. One screen (S71) settled it; it could have been part of the first round.

A replication bar that could not discriminate. Session R's "$\geq 18/20$" fails a true 0.90 recipe one time in three, and the first version of its Phase 4 reading, written by us, pooled the seeds already seen and would have detected only rates near 0.70. Both were corrected after the referee's review; Section 17 gives the rule a replication should use.

A probe that can be wrong, and is unbalanced. The split's 64-sequence probe can read a plateau as bound (R's seed 914) and weights the streams unequally, in the same way for every seed (Section 6). Neither was noticed in the project's own runs because the records do not store the probe's readings.

A distant-cue task that exceeded its oracle. Session F's first randomised header layout did not bind under the perfect gate within its budget; the lighter version with four keys per stream in two blocks did, and is the task of Section 12.

A classifier at its own threshold. Three batch-18 runs sit at $\eta^2 = 0.5$ by key and were labelled STREAM-PARTIAL; the key-first classifier labels them KEY (Section 3). The reading that depended on them is withdrawn.

Records that were not L's. A commit of "machine L's test results" held copies of X's files for 20 tests; the real files were recovered from the laptop and one test rerun (Section 15).

Branch discipline. Session I committed its first files to the main line instead of its branch; they were moved, byte-identical, and the main line cleaned. Two sessions shifted into the same seed block (1300–1359) after collisions with earlier blocks; the next round's blocks were checked against every branch before assignment.

Attribution of a number. An earlier working note put Pathway's 97.4% Sudoku figure beside TRM's 87.4% as if on one protocol; the figure is from Pathway's blog, on internal data, and the protocols differ.

Process. Container restarts cost in-flight runs on X (the test's first start), on F, G and H (while sessions were idle with only detached processes running) and on E in several batches; a detached launcher with a watcher that resumes after a restart (`longrun.py`) now runs every main-line test, and every resumed segment begins by reproducing a recorded run bit for bit. Batch 18's Muon reference did not reproduce across hosts and was rerun fresh, as the rules require.

17 Methodological findings

Revisions 5–7 recorded sixteen lessons and Revision 8 five; this revision adds seven, most of them learned from being reviewed.

Disjoint seeds make the pooled test a test. From the Muon-recipe test on, each machine has its own seeds and the pooled McNemar is the primary claim. The window-gate test's G1 is the case this was for: not shown on X alone, shown on L and pooled.

Audit the records before the next screen. The window gate had been screened, had won, and had been dropped for a reason unrelated to the obstacle at hand; one reading of the records found it. A fixed prediction before the rerun made the audit falsifiable.

Control for the bundled action. A split that copies and resets needed a reset-only arm and a copy-only arm; so does every intervention with two parts.

Score routing at the write positions. A stream's channel at key positions can differ from its channel at value positions in a bound run (session H: 34 of the balance-loss arm's 36 bound-routed runs, and two to five runs per arm elsewhere); the writes are at the values.

Borrowed devices need the model's read path. Gumbel noise is the device of a router whose reads are dense; on our routed read it collapsed routing. A balance loss balances whatever is cheapest to balance.

Have the draft checked by someone who did not write it. The first draft's fact-check was done by its author and missed seven false statements, two of them about results that were already on the branches. An adversarial referee with the records found them in a day, recomputed every number, and found the numbers right; the text was the problem. The referee now reads every main-line specification before its first run.

Fix the name of the outcome, not only the rule. A reading named "discovered" on a screen that also prints an outcome named DISCOVERED came out "does not apply" under its rule and would have applied under the outcome of the same name (4 vs 0 against 5 vs 0). Nothing was misread, but the next reader could have. A reading's outcome should be the only outcome that bears its name.

Report descriptions as descriptions. The sessions' reports, and ours, state mechanisms beside readings: that a learned write strength caused a stall, that the flat start wastes splits, that a write at the cue is what separates discovering runs. The referee found every such sentence to lack a pre-fixed rule, and some to rest on a count that no classification reproduces. They are kept, marked, and none is a result.

A replication needs a calibrated bar. "Replicated" at $\geq 18/20$ fails a true 0.90 recipe one time in three; an interval-overlap rule on pooled seeds detects only rates near 0.70. The rule for a replication of a rate r_0 on n fresh seeds is a one-sided exact test against r_0 with its size and its power at the alternative that matters stated in advance ($x \leq 46/52$ against 0.95 has size 0.045 and power 0.81 at 0.85), beside a Fisher comparison with the original count and a paired test of the mechanism's arms on the same seeds; isolation by construction, in a container that never holds the repository; records in a format the original's outcome code can score.

Store what the trigger saw. The split's decisions depend on a probe whose readings the records do not keep, so a probe that read a plateau as bound, and a probe that under-weights some streams, were found by an outsider. Every check's probe reading, the evaluation's reading at the same step and the per-stream masses belong in the record.

Branches and seeds are part of the protocol. A session that pushes to the wrong branch, a seed block that collides with another session's, and a commit of "L's results" that holds X's files are all provenance errors that cost nothing to prevent and hours to repair; each happened once.

18 Limitations

Scope of the main claim. The result is that a gate trained only on the task loss finds a partition of the memory on the *grouped layout* of a synthetic binding task, where every write's context cue is within three tokens, at two, four and eight streams, with a split aimed by gate mass at the task's key positions. The title says so. "Context-conditional binding" in general, natural data and the mechanism's role in a language model are not shown; the distant-cue task has a solution without stream labels on screens (20/20 with a hinge that reads key identities during the first 2400 updates) and no pre-registered test of it yet.

The recipe needs task structure. The split reads the key positions, and the hinge that solves the distant-cue task reads key positions and key identities; a trigger that reads all positions has not been screened. A language model has no marked keys; the analogue is untested.

The model. Three layers with shared weights (two to six screened), one head, $d = 32$ (to 128 screened), $N = 256$ (to 1024 screened), sequences of at most 99 tokens on the main line and 480 in session I's pilot, a fixed memory half-life of about 14 tokens on the main line.

Two streams. The hardest configuration relative to its size; every arm is at 65–69/80, key splits remain, and the one repair device that works at eight streams cannot work here.

Screens. Exploratory sessions use 10–40 runs per arm and lenient rules; their readings are labelled, their descriptions are descriptions, and nothing from them is a claim until a main-line test repeats it. Several screens in this revision have both arms at the ceiling and can detect only large differences.

Replication. One, from the text, by one implementer, on twenty seeds, isolated by self-report; it agrees in direction and size and is one run short of its own bar.

19 Distance to a language model

What Phase VI adds to the case. The gate is a convolution: parallel, cheap, the form every modern linear-attention model already uses. The recipe is two learning-rate groups and a plateau rule. It finds an eight-way partition without stream labels, its constants hold to four times the width and twice the depth at four streams, and its write rule can be the delta rule that the linear-attention literature already uses.

A dense loss (session I). Revision 8 conjectured that an end-of-sequence loss gives too little pressure to discover a distant cue, and that language, with a loss at every token, supplies more. Session I built a task to ask it: S sources, each with its own table of three successors per word drawn fresh for every sequence (so the sets must be learned in context), interleaved in blocks of random length and random order with a source token at each block's start, and a next-token loss at every word with a known same-source predecessor [19, 20]. Its pilot ran the perfect gate (one channel per source), one channel and a single-source control at two sources, 12,000 updates, four seeds, and could not make the task usable by its own rule, because *one channel learns to carry the source*: in three of four runs of the shorter-block configuration a step transition raised the set accuracy by 0.09–0.22 and took a linear probe of the source, from the residual entering layer 3, from 0.54–0.58 to 0.96–0.98, closing 60–80% of the gap to the perfect gate with no gate and no labels; the perfect gate's own stack never carried the source, having no need to. The sparse-loss stacks of Section 12 never did this. So the dense loss does put pressure on the stack to carry the cue, which is the conjecture's first direct evidence and is descriptive; whether a gate that reads such a stack then partitions the memory, and whether four sources are too many for one channel to tag, are the questions, and four sources need sequences of about 770 tokens, which is a GPU's job. Session I's GPU package (its branch; a device-agnostic runner for the project's models and tasks with a CPU check against a recorded run and a CUDA check at a stated tolerance) puts that screen at 9–13 hours on one consumer GPU.

What stands in the way, in order:

1. *Discovery of a distant cue without task structure.* The architecture exists and, with the key hinge, is found 20/20 (Section 12); without it the write at the cue is found on a fifth of seeds and a key split forms first on the rest. The hinge reads key identities; a dense loss may change the race without it.
2. *The split's nuisance variable.* The key positions come from the task; a trigger on all positions is the next screen.

3. *Memory horizon.* A fixed decay; the learned forget gate works on the delta write and not on the Hebbian one, and the delta write is now a candidate for the main line.

4. *Baselines.* One channel of either write rule is run at every size; Gated DeltaNet's write is the delta rule; Mixture-of-Memories' balance loss and Raven's noise were run; a residual-stream router was run. The comparison that matters is whether their routers discover *contexts* rather than token types, which none of their papers tests, and which the register result now makes askable.

A staged path. (i) On CPUs: the memory-rule test and the register test on the main line; the key-split devices; eight streams at the larger widths. (ii) On the user's GPUs, with session I's package: the dense-loss task at four sources; the recall benchmarks [18]; a replication on a second machine class. (iii) Small language models, asking one question: do the channels specialize by context or by token type.

20 Conclusion and next steps

A gate trained only on the task loss finds the partition of a Hebbian working memory that context-conditional binding needs, on the grouped layout of a synthetic task, at two, four and eight streams: a window gate, slow memory for 2400 updates, and a channel split when binding stalls, in 38 of 40 runs on two machines with disjoint seeds, where the previous recipe bound none, and in 17 of 20 in a replication from the text alone. The split's row copy is the ingredient, and the reset alone is not enough; one channel, Hebbian or delta, cannot do what the partition does at eight streams. The write rule is open: with its write strength fixed, the delta rule matches the Hebbian one everywhere tried. At a distance, a register that overwrites on the cue and gates all three layers carries it when told where to split, is found label-free on a fifth of seeds, and is found on every seed when a hinge keeps the gate off key identity during the slow phase; on Hebbian memory the first layer is the one that must be routed. Under a dense loss one channel learns to carry the cue by itself. Every number here was recomputed by a referee from the records, and the statements that were not are marked.

Next:

- L's half of `test_split_copy`, then `test_delta_recipe` on both machines after the referee's spec review, then the hinged-register test on the randomised header layout (session G's proposal: the hinged register against the baseline, the plain register and the hinge without the register; 20 seeds per machine; a rate claim of $\geq 18/20$).
- Session H's two-stream prevention (S85); session F's trigger on delta channels (S81); batch 19's eight streams at the larger sizes; session R's Phase 4.
- The dense-loss task at four sources on a GPU, once session I's package passes its CUDA check.
- Revision 10 after those; BDH-CQ and Triadic Linear Attention read in full before it.

A Provenance

test	X: spec / test commit / result commit	L: results file (commit recorded)	runs, X; L
`muon_recipe`	— / `74f5907` / `3e2d171`; pooled `2a6ba3d`	`L/muon_recipe_results.json` (`74f5907`)	156; 166
`window_gate`	spec `9a769c7`, amended `d05f0ca` / `2f98f06`, Part D `886a668` / `628eb7c`	`L/window_gate_results.json` (`886a668`; recorded `1cd20a1`)	231; 231
`split_copy`	spec `fb3acb7` / `1f52cd5` / `592c574` (X; the classifier `ba8901a`, the launcher `56c44a7`)	running	54; —
`delta_recipe`	spec in `specs/` (this revision); not written	—	—

Table 6: Phase VI main-line tests on branch `claude/bdh-growth-hebbian-inference-w90069`. L's results files for Phases III–IV are in `results/L/` from `9a53302` (twelve files, recovered from the laptop's working copies) and `9c53e16` (`channel_binding`, rerun on L on 10 October with the test's code unchanged); the Phase V files at `00ac0f2`, `d0e6c85`, the scale-axes file at `42af095`. X: Intel Xeon at 2.80 GHz for the first two tests and 2.10 GHz for `split_copy`; L: Intel i7-12650H; PyTorch 2.14.0 (L's rerun 2.14.1). L's recorded start times are local time (UTC−6); X's are UTC.

batch or session	screens (report)	commit	runs
15–18 (E)	S41–S52 (Revision 8)	`90c3a28`, `93bdfe6`, `2531480`, `7ef4e15`	379
19 (E, running)	S64 width, S65 depth; S66 eight streams and S67 capacity under way	`8cff21e`	118 at `5dfc2b5`
H	S58, S58b, S60 (reports 1–2); S61 copy without reset, S62 query gate, S63 hardness (report 3, `7e9197f`); S78 two streams (report 4, `72f0bbc`); S79 constants (report 5, `df57087`); S84 guard and evaluation trigger (report 6, `37c1fbf`); S85 running	`37c1fbf`	318; 300; 82; 62; 52
F	S53, S54, S59 (reports 1–2); S68 premise, S69 Hebbian decay, S70 lite task (report 3, `9a6c2bf`); S77 probe (report 4, `22ec912`); S76 eight streams on delta (report 5, `5b9bc2a`); S80 two and four streams (report 6, `ed76eea`); S81 running	`ed76eea`	322; 102; 30; 40; 48
G	S55 probe (report 1, `6cd36ff`); S56 register, S57 latch (report 2, `427db36`); S71 layers (report 3, `35de6db`); S73, S72 register at all layers (report 4, `93422cf`); S82 devices against the key split (report 5, `8973d48`; spec `83699a2`)	`8973d48`	22; 139; 50; 150; 136
I	S74 pilot, three configurations (report 1, `a2588da`); the GPU package (in progress)	`a2588da`	pilot
R	Phase 2 (`de8df0f`), Phase 3 diff and reruns (`00990ea`, `100dced`); Phase 4 running	`100dced`	42; 20
V	review (`d2bb959`); 8.1 check (`0fdb363`); registry (`ebb9325`); replication review (`01bee1f`); session recomputes (`4bf85e9`); L addendum (`0e6c189`)	`0e6c189`	2 reproductions

Table 7: Exploratory work and the review in Phase VI, one thread per run, each segment opened by a bit-for-bit reproduction of a recorded run; one commit per report as listed, the branch's commit column being the last report's. Branches `claude/outside-ideas`, `claude/explore-F/G/H/I`, `claude/replicate-R`, `claude/review-V`; the review is merged into the main line under `docs/review_2026-10/`. The audit is `docs/audit_2026-10.md` (`b22da04`); the reading notes and synthesis are `docs/reading/` (`ad8a92e`); the sessions' prompts are `docs/reading/parallel_sessions_prompts.md`.

B What a replication needs: the protocol as this revision states it

The items a reader must have to score runs as the main line does, compiled from session R's `UNSTATED.md` (44 entries), its `DIFF.md` (17) and the referee's additions. Where the report had been silent, the main line's choice is given.

Model. Pathway's BDH forward pass with $x = \mathrm{LN}(\mathrm{emb})$, per layer $x_s = \mathrm{ReLU}(xE)$, $y = \mathrm{LN}(\mathrm{attn}(x_s, x_s, x))$, $y_s = \mathrm{ReLU}(yE_v)$, one head, $N = 256$, $d = 32$, three applications of one shared layer; scalar decay 0.95^{t-s} in place of RoPE; strictly causal ($s < t$); no dropout; Pathway's initialisation (encoder, decoder, head and embedding at $\mathcal{N}(0, 0.02^2)$), which sets the scale of the gate's input; channel mixing as the score $(x_t \cdot x_s)(g_t \cdot g_s)\, 0.95^{t-s}$ with one gate per token for read and write, the same at every layer; the gate reads the model's own raw embedding (the same table), through a width-3 depthwise causal convolution (weights uniform in $\pm 1/\sqrt{3}$, no bias, left-padded with zeros), a hidden layer of 32 units with tanh, $W_{\mathrm{in}}, W_g \sim \mathcal{N}(0, 0.1^2)$, no biases; the "conv" of the four- and eight-stream configurations is one shared depthwise causal filter of width 4 on the d-dimensional layer input, initialised to the identity, whose output feeds the keys (equal to the queries) and the values.

Task. 28 tokens at eight streams and four keys: 16 value tokens, P key tokens, S context tokens, and no query token (the sequence of 99 tokens ends with the query's value; the model reads the first 98 and predicts the 99th from the query's KEY position); per sequence and key, S distinct values without replacement, assigned to streams in random order; the query's stream and key uniform and independent; the loss is cross-entropy over the full vocabulary at the query's KEY position; accuracy is argmax over the full vocabulary; 2048 held-out sequences from a fixed generator, the same for every run and arm; `torch.manual_seed(seed)` before initialisation, the training batches from `Generator(seed + 10000)`.

Training and evaluation. Adam with default betas, no weight decay, batch 32, constant rates, no warm-up, no clipping; SLOW's groups and its switch after the evaluation at 2400; evaluation every 1200 updates and at the budget; early stop after three consecutive evaluations ≥ 0.95; at a shared step the order is evaluation, statistics, the path's own check, the learning-rate switch, the split trigger, the early-stop test; the perfect gate trains at the single rate 10^{-3} without SLOW; the final per-stream accuracy and map come from a final evaluation of the final model.

Outcomes. Bound: the first evaluation ≥ 0.95 that holds to the end (a run can bind on its last evaluation); the stream-to-channel map: for each stream, the argmax over channels of the mean gate at the body's value positions on a 256-sequence probe (`Generator(seed + 99000)`), ties on unrounded values; bound routed: bound, the map one-to-one, every stream's accuracy ≥ 0.9 on the held-out queries of that stream; failure labels as Section 3.

The split. The probe: 64 sequences from `BindTask.make_batch(64, Generator(12345))`, the same for every seed and arm, scored on their queries (5, 5, 6, 8, 10, 12, 5 and 13 per stream at eight streams); read at 2400 as the first reference and at every check from 4800 in steps of 2400 up to the budget minus 2400; fire if the probe's accuracy is below 0.95 and rose by less than 0.02 since the previous check; c^* and c_0 by the mean read-gate mass over the probe's 32 body key positions (not the query's); $W_g[c_0] \leftarrow W_g[c^*]$ (the main line's Part D form adds $0.1\,\sigma$ noise to both rows, σ the copied row's standard deviation, c^*'s noise drawn first from `Generator(12_000_000 + 1000*seed`

+ n_splits), and zeroes W_g's Adam state including its step counter); a split blocks further splits for 4800 updates; at most three; no guard against a target that carries key mass (none is needed at $k = 16$, $S = 8$).

Records. Per run: the curve at every evaluation, the per-stream gate mass at value positions and per-stream accuracy at the end, each check's probe reading and the evaluation's reading at the same step, each split's update, c^*, c_0 and the map before it, the CPU, the code commit and the start time with its time zone.

Statistics. Exact one-sided McNemar on the discordant pairs, $p = P[\mathrm{Bin}(b + c, \frac{1}{2}) \geq b]$; Wilson 95% intervals with $z = 1.96$; for a replication of a rate on n fresh seeds, a one-sided exact test against the reported rate with its size and power stated in advance, a Fisher comparison with the original count, and the mechanism's arms paired on the same seeds.

References

[1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain.* arXiv:2509.26507, 2025.

[2] B. Engdahl, A. Kosowski, J. Chorowski, Z. Stamirowska, P. Uznański, J. Jiang, R. Phadke, R. Kinas, R. Zhong. *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning.* arXiv:2608.09888, 2026.

[3] O. Sieberling et al. *Triadic Linear Attention.* arXiv:2609.36529, 2026.

[4] A. Gu, T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752, 2023.

[5] K. A. Wang, J. Shi, E. B. Fox. *Test-time regression: a unifying framework for designing sequence models with associative memory.* arXiv:2501.12352, 2025.

[6] S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule.* arXiv:2412.06464, 2025.

[7] A. Jolicoeur-Martineau. *Less is More: Recursive Reasoning with Tiny Networks.* arXiv:2510.04871, 2025.

[8] J. Geiping et al. *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach.* arXiv:2502.05171, 2025.

[9] T. Flesch, D. G. Nagy, A. Saxe, C. Summerfield. *Modelling continual learning in humans with Hebbian context gating and exponentially decaying task signals.* arXiv:2203.11560, 2022.

[10] A. Iyer et al. *Avoiding Catastrophe: Active Dendrites Enable Multi-Task Learning in Dynamic Environments.* arXiv:2201.00042, 2022.

[11] M. Choraria et al. *Context-Gated Associative Retrieval: From Theory to Transformers.* arXiv:2605.10970, 2026.

[12] J. Du et al. *MoM: Linear Sequence Modeling with Mixture-of-Memories.* arXiv:2502.13685, 2025.

[13] A. Afzal, A. Bick et al. *Raven: High-Recall Sequence Modeling with Sparse Memory Routing.* arXiv:2607.25357, 2026.

[14] A. Basu. *When Does Content-Based Routing Work? Representation Requirements for Selective Attention in Hybrid Sequence Models.* arXiv:2603.20997, 2026.

[15] J. Chung, S. Ahn, Y. Bengio. *Hierarchical Multiscale Recurrent Neural Networks*. arXiv:1609.01704, 2017.

[16] A. Behrouz et al. *Nested Learning: The Illusion of Deep Learning Architectures*. arXiv:2512.24695, 2025.

[17] W. Fedus, B. Zoph, N. Shazeer. *Switch Transformers*. JMLR 23(120), 2022.

[18] S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models*. arXiv:2312.04927, 2023.

[19] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jegou, L. Bottou. *Birth of a Transformer: A Memory Viewpoint*. arXiv:2306.00802, 2023.

[20] B. L. Edelman, E. Edelman, S. Goel, E. Malach, N. Tsilivis. *The Evolution of Statistical Induction Heads: In-Context Learning Markov Chains*. arXiv:2402.11004, 2024.